

Angular Avançado · Módulos 3, 6 e 7

Sessão 10

Guards, interceptors e Signal Forms

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 9

Stores, AuthStore e o primeiro guard

FlightStore

- Estado no root: sobrevive à navegação
- `linkedSignal` com `set`: uma janela sobre a store
- `debounce` no schema do form

AuthStore

- A mesma receita: `_userName` + `asReadonly()`
- `isLoggedIn` e `token` como `computed`
- Login / Logout na navbar

canActivate

- `authGuard` no `flight-edit/:id`
- `false` cancela · `UrlTree` redireciona
- `?needsLogin=true` → `input()` na Home

Falta: a rota **conta** para este utilizador? (`canMatch`) · pode **sair**? (`canDeactivate`)

Agenda de hoje

01	**Guards (continuação)**	canMatch · canDeactivate	≈ 27 min
02	**Interceptors funcionais**	Juntar o token · a ordem da cadeia	≈ 25 min
03	**Signal Forms**	Carregar · validar · mostrar erros · gravar	≈ 56 min
04	**Resumo**	E o que vem na Sessão 11	≈ 5 min



Bloco A · Guards

Guards funcionais (continuação)

A rota conta? Pode sair?

Quatro tipos de guard

Recap: o canActivate ficou feito na Sessão 9

Propriedade da rota	Tipo	Pergunta	Exemplo
`canActivate`	CanActivateFn	Pode **entrar**?	Só com sessão
`canActivateChild`	CanActivateChildFn	Pode entrar nas **filhas**?	Toda uma área de admin
`canMatch`	CanMatchFn	Esta rota **conta** para este URL?	Rotas por perfil · feature flags
`canDeactivate`	CanDeactivateFn<T>	Pode **sair**?	Alterações por guardar

Todos são **arrays**: a navegação só continua se **todos** os guards da rota disserem que sim.



Da teoria ao código

Agora, vamos escrever o código

- `canMatch`: a mesma rota `boarding`, dois componentes
- `canDeactivate`: avisar antes de perder alterações no FlightEdit
- G5, G6 e a variante `returnUrl`: só nos slides

shared/util-auth/auth-guard.ts · shared/util-common/exit-guard.ts · ticketing.routes.ts

canMatch: a rota só existe para alguns

```
// auth-guard.ts
export const authMatch: CanMatchFn = () =>
  inject(AuthStore).isLoggedIn();

// ticketing.routes.ts — o mesmo path duas vezes; a ordem conta
{ path: 'boarding', component: BoardingList, canMatch: [authMatch] },
{ path: 'boarding', component: BoardingLocked },   // a "seguinte"
```

	canActivate a false	canMatch a false
O que faz o router	cancela a navegação	ignora a rota e **continua a procurar**
Na Skyboard	fica onde estava	mostra a `BoardingLocked` — **mesmo URL**
Bom para	"precisas de login"	a mesma rota com versões diferentes (perfil, flags)

O `canMatch` não impede o preloading: decide o **match**, não o download.

canDeactivate: antes de perder alterações

```ts
// shared/util-common/exit-guard.ts
export interface DirtyAware {             // o contrato
  isDirty(): boolean;
}

export const exitGuard: CanDeactivateFn<DirtyAware> = (component) => {
  if (!component.isDirty()) {
    return true;
  }
  return confirm('Do you really want to leave without saving?');
};
```

Notar

- Recebe a **instância** do componente
- Uma `interface` chega: aqui é só um **tipo**, não um token de DI
- `confirm()`: para a demo

Na rota: `canDeactivate: [exitGuard]` — ao lado do `canActivate: [authGuard]`.

O FlightEdit diz se tem alterações

feature-booking/flight-edit/flight-edit.ts

```ts
export class FlightEdit implements DirtyAware {
  readonly id = input.required({ transform: numberAttribute });
  readonly showDetails = input(false, { transform: booleanAttribute });

  // por agora simulado; com as Signal Forms: this.flightForm().dirty()
  protected readonly dirty = signal(false);

  isDirty(): boolean {
    return this.dirty();
  }
}
```

Experimentar

- Login → **Edit**
- **Simulate a change**
- **Back to search** → pergunta
- Sem alterações → sai logo

No template: `<button (click)="dirty.set(true)">Simulate a change</button>`. Sai o código de demo da Sessão 7.

Na vida real: um diálogo em vez do confirm()

Referência — um guard assíncrono: o router espera

```ts
export const exitGuard: CanDeactivateFn<DirtyAware> = (component) => {
  if (!component.isDirty()) {
    return true;
  }
  const dialogRef = inject(Dialog).open<boolean>(ConfirmDialog, {
    data: 'Do you really want to leave without saving?',
  });
  return dialogRef.closed.pipe(map((result) => result === true));
};
```

```ts
// confirm-dialog.ts
message =
inject<string>(DIALOG_DATA);
dialogRef =
inject<DialogRef<boolean>>(
  DialogRef);

close(decision: boolean) {
  this.dialogRef.close(decision);
}
```

`DIALOG_DATA`: quem cria o componente dá-lhe um injector com os dados (ver o anexo: um Injector à medida).

Código antigo: guards em classe

Referência — ligar sem reescrever, com mapTo…

antes · legacy-auth-guard.ts

```ts
@Injectable({ providedIn: 'root' })
export class LegacyAuthGuard implements CanActivate {
  private readonly auth = inject(AuthStore);
  private readonly router = inject(Router);

  canActivate(route, state): boolean | UrlTree {
    return this.auth.isLoggedIn()
      || this.router.createUrlTree(['/home']);
  }
}
```

agora · ticketing.routes.ts

```ts
// rota: liga a classe sem a reescrever
canActivate: mapToCanActivate([LegacyAuthGuard])

// o que faz por dentro
canActivate: [(route, state) =>
  inject(LegacyAuthGuard).canActivate(route, state)]

// misturar com guards funcionais
[...mapToCanActivate([LegacyAuthGuard]), authGuard]
```

Família: `mapToCanActivate` · `mapToCanActivateChild` · `mapToCanDeactivate` · `mapToCanMatch` · `mapToResolve`

Código novo: **funções** (G2–G4). Código existente: `mapTo…` e migrar aos poucos.

Variante: voltar onde estava

Referência — Login na mensagem da Home + returnUrl

```ts
// auth-guard.ts — state.url = o URL bloqueado
export const authGuard: CanActivateFn = (route, state) => {
  …
  return router.createUrlTree(['/home'], {
    queryParams: { needsLogin: true, returnUrl: state.url },
  });
};
```

```ts
// home.ts
readonly returnUrl = input<string>();

protected login(): void {
  this.auth.login('Maria');
  this.router.navigateByUrl(
    this.returnUrl() ?? '/home');
}
```

```html
<!-- home.html -->
@if (needsLogin()) {
  <p><strong>Please log in to continue.</strong>
     <button (click)="login()">Login</button></p>
}
```

Porquê

- Só o botão não chega: o `needsLogin` fica no URL
- Validar que o `returnUrl` é **interno**

Guards: resumo

canActivate

- Pode **entrar**?
- `false` cancela · `UrlTree` redireciona

canMatch

- A rota **conta**?
- `false` → o router continua à procura

canDeactivate

- Pode **sair**?
- Recebe a instância do componente

Funções num **injection context** → `inject()`. Ajudam a UX; a segurança está na API.

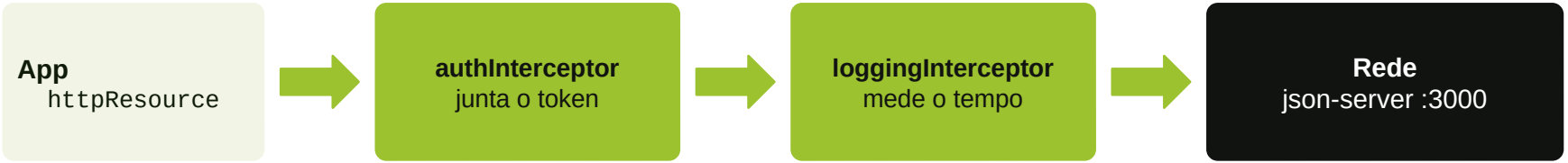


Bloco B · Interceptors

Interceptors funcionais

Funções entre a app e a rede

Uma cadeia entre a app e a rede



`pedido →` `← resposta (ordem inversa)`

- Vê **todos** os pedidos do `HttpClient` — incluindo os do `httpResource`
- Cada um recebe o pedido e o `next`: mexe no pedido e passa-o ao seguinte
- Usos típicos: **token**, erros `401`/`403` num só sítio, logging, loading global, retry, cache

Anatomia de um interceptor

```ts
export const myInterceptor: HttpInterceptorFn = (req, next) => {
  const store = inject(AuthStore);              // injection context

  // ANTES do next: o pedido (imutável → clone)
  const newReq = req.clone({ setHeaders: { 'X-App': 'skyboard' } });

  // o next: passa ao seguinte da cadeia
  return next(newReq).pipe(
    // DEPOIS do next: a resposta (tap, catchError, finalize…)
    catchError((error) => { /* … */ return throwError(() => error); }),
  );
};
```

O `HttpClient` é baseado em **Observables** — os interceptors também, mesmo quando a app usa `httpResource`.

Registar: provideHttpClient(withInterceptors(…))

```
// app.config.ts
providers: [
  provideRouter(routes, …),
  provideHttpClient(
    withInterceptors([authInterceptor, loggingInterceptor]),
  ),
  provideLanguageService('browser'),
],
```

É uma **provider function** — o mesmo padrão do `provideLanguageService` da Sessão 8.

Notar

- Sem `provideHttpClient` o `HttpClient` já existe (Angular 22) — mas para **configurar** é preciso
- A ordem do array **é** a ordem da cadeia
- Código antigo (classes): `withInterceptorsFromDi()`



Da teoria ao código

Agora, vamos escrever o código

- Um interceptor que junta o token da `AuthStore`
- Um segundo, de logging — e trocar a ordem para ver a cadeia
- Ver os headers na Network e os logs na consola

shared/util-auth/auth-interceptor.ts · shared/util-common/logging-interceptor.ts · app.config.ts

Juntar o token aos pedidos da nossa API

domains/shared/util-auth/auth-interceptor.ts

```ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthStore).token();
  const apiUrl = inject(ConfigService).apiUrl;

  // sem sessão, ou pedido para outro servidor: segue tal como está
  if (!token || !req.url.startsWith(apiUrl)) {
    return next(req);
  }

  // os pedidos são imutáveis: clona-se com o header novo
  const authReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  return next(authReq);
};
```

O interceptor vê **todos** os pedidos da app. O `startsWith(apiUrl)` garante que o token não vai para terceiros.

Registar e ver na Network

```
// app.config.ts
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './domains/shared/util-auth/auth-interceptor';

provideHttpClient(withInterceptors([authInterceptor])),
```

Sem sessão

- Network → `/flights` → Request Headers
- Sem `Authorization`

Com sessão

- **Login → Search**
- `Authorization: Bearer demo-token-maria`

Um segundo interceptor: a ordem conta

shared/util-common/logging-interceptor.ts

```ts
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const started = performance.now();
  const auth = req.headers.has('Authorization') ? 'com token' : 'sem token';
  console.log(`→ ${req.method} ${req.urlWithParams} (${auth})`);

  return next(req).pipe(                // depois do next: a resposta
    finalize(() => {
      const ms = Math.round(performance.now() - started);
      console.log(`← ${req.method} ${req.urlWithParams} · ${ms} ms`);
    }),
  );
};
```

[auth, logging]

- Consola: `(com token)`

[logging, auth]

- Consola: `(sem token)`

A ordem do array é a ordem no **pedido**; na resposta é a inversa.

Para saber: erros 401/403 num só sítio

```ts
export const authErrorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthStore);
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === HttpStatusCode.Unauthorized
          || error.status === HttpStatusCode.Forbidden) {
        auth.logout();
        router.navigate(['/home'], { queryParams: { needsLogin: true } });
      }
      return throwError(() => error);   // quem pediu também sabe
    }),
  );
};
```

Outros usos

- Loading global (contar pedidos ativos)
- `retry`
- Cache



Bloco C · Signal Forms

Signal Forms: o essencial

Carregar, validar, mostrar erros e gravar — no FlightEdit

Três peças: modelo, form() e [formField]

Modelo
um
`WritableSignal<Flight>`

→

`form(modelo)`
→ um
`FieldTree`

→

`[formField]`
liga um campo a um input

```
// já o fizeram no FlightSearch (Sessão 2)
protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });
protected readonly filterForm = form(this.filter);

<input [formField]="filterForm.from" />
```

O formulário **não copia** os dados: é uma vista sobre o signal. Escrever no input = escrever no modelo.

O FieldTree: campos e estado

```
flightForm.from              // o CAMPO  → [formField]="flightForm.from"
flightForm.from()            // o ESTADO do campo
flightForm.from().value()    // 'Lisbon'
flightForm.from().errors()   // [{ kind: 'required' }]

flightForm()                 // o estado do formulário inteiro
flightForm().dirty()         // algum campo mudou?

flightForm.aircraft.type     // a árvore segue o modelo
flightForm.prices[0].amount
```

Estado = signals

- `value`
- `dirty` · `touched`
- `valid` · `invalid` · `errors`
- `pending` (async)
- `disabled` · `hidden` · `readonly`

Sem parênteses: o **campo**. Com parênteses: o **estado**.

O schema: regras e comportamento

```ts
form(this.flight, (path) => {
  required(path.from, { message: 'From is required' });
  minLength(path.from, 3);
  debounce(path.from, 300);                    // Sessão 9
  hidden(path.delay, { when: (ctx) => !ctx.valueOf(path.delayed) });
});

// num ficheiro próprio: as regras do domínio
export const flightSchema = schema<Flight>((path) => { … });
```

Regras

- `required` · `min`/`max`
- `minLength` · `pattern` · `email`
- `validate()` à medida

Comportamento

- `debounce` · `hidden`
- `disabled` · `readonly`

O `path` são **caminhos** do modelo, não valores. Compor schemas: `apply(path, outroSchema)`.

Submit: a opção submission + [formRoot]

```ts
form(this.flight, flightSchema, {
  submission: {
    action: () => this.save(),        // só se for válido
    onInvalid: (form) => /* foco no 1.º erro */,
  },
});
```

```html
<form [formRoot]="flightForm">
  …
  <button>Save</button>
</form>
```

Notar

- A `action` devolve `null` (ok) ou **erros** — ficam no formulário
- `formRoot`: `preventDefault`, `novalidate`, Enter
- `ignoreValidators`: `'pending'` (default) · `'none'` · `'all'`

Um erro do servidor aparece como os erros de validação.

Typed Forms vs Signal Forms

Vão encontrar os dois — os conceitos mapeiam

	Reactive / Typed Forms	Signal Forms
Onde vive o estado	nos `FormControl` / `FormGroup`	num **signal** (o modelo)
Reagir a mudanças	`valueChanges` (Observable)	signals: `value()`, `dirty()`…
Validação	`Validators` em cada controlo	um **schema** separado do modelo
Ligar ao template	`[formGroup]` · `formControlName`	`[formField]` · `[formRoot]`
Controlo à medida	`ControlValueAccessor`	`model()` (`FormValueControl`)

Os dois convivem: não há pressa em migrar código existente.



Da teoria ao código

Agora, vamos escrever o código

- Carregar o voo do servidor e ligar o formulário — o `exitGuard` passa a ser real
- Regras, erros e um schema num ficheiro próprio
- Gravar com o submit (`PUT`, pelos interceptors)

data/flight-client.ts · data/flight-schema.ts · feature-booking/flight-edit/*

F1 · Carregar o voo e ligar o formulário

```
// flight-client.ts
findByIdResource(id: Signal<number>) {
  return httpResource<Flight>(() => `${this.config.apiUrl}/${id()}`, {
    defaultValue: initialFlight,
  });
}
```

```
feature-booking/flight-edit/flight-edit.ts
export class FlightEdit implements DirtyAware {
  readonly id = input.required({ transform: numberAttribute });
  private readonly flightResource = this.flightClient.findByIdResource(this.id);

  // a cópia local editável (como o filter da Sessão 9)
  protected readonly flight = linkedSignal(() => {
    const flight = this.flightResource.value();
    return { ...flight, date: toLocalInput(flight.date) };
  });
  protected readonly flightForm = form(this.flight);

  isDirty(): boolean { return this.flightForm().dirty(); }   // o G4, agora real
}
```

F1 · O template

```
<h1>Flight Edit #{{ id() }}</h1>
<form>
  <fieldset>
    <input id="from" class="form-control" [formField]="flightForm.from" />
    <input id="to" class="form-control" [formField]="flightForm.to" />
    <input type="datetime-local" [formField]="flightForm.date" />
    <input type="checkbox" [formField]="flightForm.delayed" />
    <input type="number" [formField]="flightForm.delay" />
  </fieldset>
</form>

<!-- debug -->
dirty: {{ flightForm().dirty() }} · invalid: {{ flightForm().invalid() }}
<pre>{{ flight() | json }}</pre>
```

Experimentar

- Login → **Edit**
- Network: `GET /flights/5` com token
- Mudar o From → `dirty: true`
- **Back to search** → o `confirm()` do G4

`type="number"` e checkbox: no modelo, `delay` continua `number` e `delayed` `boolean`.

F2 · Primeiras regras e o estado à vista

```
protected readonly flightForm = form(this.flight, (path) => {
  required(path.from);
  required(path.to);
  required(path.date);
  minLength(path.from, 3);
});

<!-- por baixo do From -->
<pre>{{ flightForm.from().errors() | json }}</pre>
```

```
// From = 'Fa'
[{ "kind": "minLength",
   "minLength": 3 }]

// From = ''
[{ "kind": "required" }]
```

Campo vazio não dispara o `minLength`: os campos são opcionais — quem é obrigatório tem `required`.

F3 · O schema num ficheiro próprio + mensagens

domains/ticketing/data/flight-schema.ts

```ts
export const flightSchema = schema<Flight>((path) => {
  required(path.from, { message: 'From is required' });
  minLength(path.from, 3, { message: 'From needs at least 3 characters' });
  required(path.to, { message: 'To is required' });
  required(path.date, { message: 'Date is required' });
});
```

```ts
// flight-edit.ts
protected readonly flightForm =
  form(this.flight, flightSchema);
```

```html
<!-- por baixo do From, do To e da Date -->
@for (error of flightForm.from().errors(); track $index) {
  <div class="validation-errors">{{ error.message }}</div>
}
```

Regras do **domínio** → pasta `data`. O `@for` repetido vira um componente na Sessão 11.

F4 · Gravar

```ts
// flight-client.ts
save(flight: Flight): Promise<Flight> {
  return firstValueFrom(this.http.put<Flight>(`${this.config.apiUrl}/${flight.id}`, flight));
}
```

```ts
feature-booking/flight-edit/flight-edit.ts
protected readonly flightForm = form(this.flight, flightSchema, {
  submission: {
    action: () => this.save(),
    onInvalid: (flightForm) => flightForm().errorSummary()[0]?.fieldTree().focusBoundControl(),
  },
});

private async save(): Promise<TreeValidationResult> {
  try {
    const flight = this.flight();
    await this.flightClient.save({ ...flight, date: new Date(flight.date).toISOString() });
    this.flightForm().reset();          // volta a pristine: o exitGuard deixa sair
    this.message.set('Saved');
    return null;
  } catch {
    return { kind: 'server', message: 'Could not save the flight' };
  }
}
```

F4 · Submeter e experimentar

```
<form [formRoot]="flightForm">
  <fieldset> … </fieldset>

  <!-- erros do formulário (ex.: do servidor) -->
  @for (error of flightForm().errors(); track $index) {
    <div class="validation-errors">{{ error.message }}</div>
  }

  <button>Save</button>
  <small>{{ message() }}</small>
</form>
```

Experimentar

- From vazio → **Save** → foco no From, nada enviado
- "Funchal" → **Save** → `PUT` com token · "Saved"
- **Back to search** → sai sem perguntar
- json-server parado → "Could not save the flight"

O `PUT` passa pelos interceptors; o `reset()` deixa o `exitGuard` sossegado. A sessão inteira no mesmo ecrã.

Fica para a Sessão 11

Erros num componente

- `shared/ui-forms`
- Substitui os `@for` repetidos

hidden + condicional

- `delay` só com `delayed`
- `applyWhenValue` · `min(…, 15)`

Uma regra nossa

- From ≠ To com `validate()`
- `errorSummary()`

Mais tarde, se pedirem: validadores async · arrays (`prices`) · Zod · campos à medida.

Resumo da sessão

Guards

- Entrar? · a rota conta? · sair?
- `canMatch` a `false` → a rota seguinte
- `canDeactivate` recebe o componente

Interceptors

- `(req, next) · clone()`
- `provideHttpClient(withInterceptors([…]))`
- A ordem do array é a cadeia

Signal Forms

- Modelo → `form()` → `FieldTree`
- Schema: regras + comportamento
- `submission` + `[formRoot]`

O FlightEdit junta tudo: `authGuard` + `exitGuard` com o `dirty()` real + `GET/PUT` pelos interceptors.



Na Sessão 11

Forms (fim) e arquitetura

- Erros num componente · `hidden` · uma regra From ≠ To
- Domínios e camadas: feature · ui · data · util
- Regras de dependência e facades